

1 C语言程序基础

大纲

- 程序入口**：C语言程序必须从 `main()` 函数开始执行，且有且仅有一个主函数
- 注释方式**：
 - 多行注释： `/* 内容 */`
 - 单行注释： `// 内容`
- 语句结束**：每条执行语句必须以英文分号 `;` 结尾
- 程序结构**：头文件引入 → 全局定义 → main函数 → 执行语句 → return返回
- 基本输入输出**：使用 `printf()` 输出，需引入 `stdio.h` 头文件
- 大小写敏感**：C语言严格区分大小写字母
- 字符串表示**：字符串必须用英文双引号括起来

1.1 知识点总结

1.1.1 最小程序结构

```
#include <stdio.h>          /* 预处理：包含头文件 */

int main(void) {            /* 主函数，程序入口 */
    printf("Hello, C!\n");
    return 0;               /* 返回 0 表示正常结束 */
}
```

1.1.2 运行流程

阶段	说明	常见产物
预处理	处理 <code>#</code> 指令 (include、define 等)	<code>.i</code> 文件
编译	将 C 代码翻译为汇编	<code>.s</code> 文件
汇编	汇编代码转为机器码	<code>.o</code> / <code>.obj</code> 目标文件
链接	合并目标文件与库，生成可执行文件	<code>.exe</code> / 无后缀可执行文件

1.1.3 源文件与头文件

类型	扩展名	作用
源文件	<code>.c</code>	存放函数实现
头文件	<code>.h</code>	存放声明、宏、类型定义，供 <code>#include</code>

1.1.4 注释

方式	语法	说明
单行	<code>// 注释</code>	C99 起支持
单行	<code>/* 注释 */</code>	传统写法
多行	<code>/* 多行\n注释 */</code>	不能嵌套

1.1.5 命名规则 (重点)

规则	说明	合法示例	非法示例
首字符	字母或 <code>_</code>	<code>_count</code> , <code>sum</code>	<code>2abc</code>
后续字符	字母、数字、 <code>_</code>	<code>var_1</code>	<code>a-b</code>
区分大小写	<code>Sum</code> 与 <code>sum</code> 不同	—	—
不能是关键字	如 <code>int</code> 、 <code>return</code>	<code>my_int</code>	<code>int</code>

1.1.6 关键字 (常用)

类别	关键字
类型	<code>char</code> <code>int</code> <code>float</code> <code>double</code> <code>void</code> <code>short</code> <code>long</code> <code>signed</code> <code>unsigned</code>
控制流	<code>if</code> <code>else</code> <code>switch</code> <code>case</code> <code>default</code> <code>for</code> <code>while</code> <code>do</code> <code>break</code> <code>continue</code> <code>goto</code>
函数/存储	<code>return</code> <code>static</code> <code>extern</code> <code>register</code> <code>auto</code> <code>const</code> <code>volatile</code>
结构	<code>struct</code> <code>union</code> <code>enum</code> <code>typedef</code>
其他	<code>sizeof</code> <code>break</code> <code>continue</code>

1.1.7 表达式 (重点)

概念	说明	示例
表达式	有值的式子	<code>a + 1</code> 、 <code>x > 0</code>
语句	以 <code>;</code> 结尾，执行动作	<code>a = 5;</code> 、 <code>printf("hi");</code>
复合语句	用 <code>{ }</code> 包裹多条语句	<code>if (x) { a=1; b=2; }</code>

1.1.8 主要特点（速记）

特点	说明
面向过程	以函数为组织单位
编译型	先编译再运行，效率高
贴近硬件	可指针操作、位运算
可移植	标准 C 代码跨平台性好
无自动内存管理	动态内存需手动 <code>free</code>
弱类型检查	数组不检查边界，需程序员注意

2 数据类型与常量

大纲

- 1. 三大基本数据类型：
 - 整型：`int`
 - 浮点型：`float`（单精度）、`double`（双精度）
 - 字符型：`char`（占用1字节）
- 2. 常量定义：使用 `#define` 预处理指令定义常量，如 `#define PI 3.14159`
- 3. 标识符规则：
 - 只能由字母、数字、下划线组成
 - 不能以数字开头
 - 不能使用关键字
- 4. 运算符：
 - `/`：除法（整数相除取整）
 - `%`：取余（仅适用于整型）
- 5. 格式化输出：
 - `%d`：整型格式符
 - `%f`：浮点格式符

- o `%c`：字符格式符

2.1 知识点总结

2.1.1 基本数据类型（重点）

类型	含义	典型大小（32/64位）	格式符
<code>char</code>	字符/小整数	1 字节	<code>%c</code>
<code>short</code>	短整型	2 字节	<code>%hd</code>
<code>int</code>	整型	4 字节	<code>%d</code>
<code>long</code>	长整型	4/8 字节	<code>%ld</code>
<code>long long</code>	更长整型	8 字节	<code>%lld</code>
<code>float</code>	单精度浮点	4 字节	<code>%f</code>
<code>double</code>	双精度浮点	8 字节	<code>%lf</code> (scanf) / <code>%f</code> (printf)
<code>void</code>	无类型	—	用于函数返回值或通用指针

2.1.2 类型修饰符

修饰符	作用	示例
<code>signed</code>	有符号（默认）	<code>signed int x;</code>
<code>unsigned</code>	无符号，只表示非负数	<code>unsigned int n;</code>
<code>short</code>	缩短整型	<code>short int s;</code>
<code>long</code>	加长整型	<code>long int l;</code>
<code>const</code>	只读，不可修改	<code>const int MAX = 100;</code>

2.1.3 变量声明与初始化（重点）

写法	说明	示例
声明	指定类型和名字	<code>int age;</code>
初始化	定义时赋初值	<code>int x = 10;</code>
多变量	同类型一次声明	<code>int a = 1, b = 2;</code>
局部变量	未初始化值不确定	<code>int x;</code>
全局/静态	未初始化默认为 0	<code>static int count;</code>

2.1.4 整型常量写法 (重点)

进制	前缀	示例	实际值
十进制	无	<code>255</code>	255
八进制	<code>0</code>	<code>0377</code>	255
十六进制	<code>0x</code> / <code>0X</code>	<code>0xFF</code>	255
长整型后缀	<code>L</code> / <code>l</code>	<code>100L</code>	—
无符号后缀	<code>U</code> / <code>u</code>	<code>100U</code>	—

十进制：0 1 2 3 4 5 6 7 8 9 二进制：0 1 八进制：0 1 2 3 4 5 6 7 十六进制：0 1 2 3 4 5 6 7 8 9 A B C D E F 方法：短除法（进制转换）

2.1.5 浮点常量 (重点)

写法	说明	示例
小数形式	含小数点	<code>3.14</code>
指数形式	e/E 表示 10 的幂	<code>1.2e-3</code> → 0.0012
单精度后缀	<code>f</code> / <code>F</code>	<code>3.14f</code>
长双精度后缀	<code>L</code>	<code>3.14L</code>

2.1.6 字符与字符串常量 (重点)

类型	定界符	示例	说明
字符	单引号 <code>' '</code>	<code>'A'</code> 、 <code>'\n'</code>	占 1 字节（存储）
字符串	双引号 <code>" "</code>	<code>"Hello"</code>	末尾自动加 <code>\0</code>

2.1.7 常用转义字符 (重点)

转义	含义
<code>\n</code>	换行
<code>\t</code>	制表符
<code>\\</code>	反斜杠
<code>\'</code>	单引号
<code>\"</code>	双引号
<code>\0</code>	空字符（字符串结束）
<code>\ddd</code>	八进制 ASCII 码
<code>\xhh</code>	十六进制 ASCII 码

2.1.8 常量定义方式 (重点)

方式	语法	特点
宏常量	<code>#define PI 3.14</code>	预处理替换，无类型
const 变量	<code>const double PI = 3.14;</code>	有类型，更安全
枚举	<code>enum { RED, GREEN, BLUE };</code>	整型常量集

2.1.9 sizeof 求类型/变量大小 (重点)

用法	示例	结果（常见）
类型	<code>sizeof(int)</code>	4
变量	<code>sizeof(x)</code>	取决于 x 的类型
字符	<code>sizeof(char)</code>	1
数组	<code>sizeof(a)</code>	元素个数 × 元素大小

2.1.10 类型转换 (重点)

类型	说明	示例
隐式转换	编译器自动转换	<code>int x = 3.9;</code> → <code>x=3</code>
显式转换	强制类型转换	<code>(int)3.9</code> 、 <code>(float)n</code>
整型提升	char/short 运算时提升为 int	<code>'a' + 1</code>

2.1.11 取值范围 (典型, 有符号 int)

类型	约略范围
<code>char</code>	-128 ~ 127 或 0 ~ 255 (unsigned)
<code>short</code>	-32768 ~ 32767
<code>int</code>	约 ±21 亿
<code>unsigned int</code>	0 ~ 约 42 亿

3 运算符

大纲

- 1. 取余运算符: `%` 仅适用于整型数据, 返回除法的余数
- 2. 自增运算符:
 - `i++`: 后置自增, 先使用当前值, 再自增1
 - `++i`: 前置自增, 先自增1, 再使用新值
- 3. 复合赋值运算符:
 - `a += b` → `a = a + b`
 - `a -= b` → `a = a - b`
 - `a *= b` → `a = a * b`
 - `a /= b` → `a = a / b`
 - `a %= b` → `a = a % b`
- 4. 运算符优先级:
 - 最高: `()` (括号)
 - 算术运算符: `*`、`/`、`%` 高于 `+`、`-`
 - 赋值运算符优先级最低
- 5. 格式化输入: `scanf("%lf", &r)` 用于读取double类型数据

3.1 知识点总结

3.1.1 算术运算符（重点）

运算符	含义	示例	注意
<code>+</code>	加	<code>a + b</code>	—
<code>-</code>	减	<code>a - b</code>	—
<code>*</code>	乘	<code>a * b</code>	—
<code>/</code>	除	<code>a / b</code>	整数/整数 → 整数（截断）
<code>%</code>	取模	<code>a % b</code>	仅用于整型

整数除法示例：`5 / 2 = 2` , `5.0 / 2 = 2.5`

3.1.2 关系运算符（重点）

运算符	含义	结果类型
<code>></code> <code><</code> <code>>=</code> <code><=</code>	比较大小	真为 1，假为 0
<code>==</code>	等于	注意与赋值 <code>=</code> 区分
<code>!=</code>	不等于	—

3.1.3 逻辑运算符（重点）

运算符	含义	示例	短路				
<code>&&</code>	逻辑与	<code>a && b</code>	左假则不算右				
<code>`</code>		<code>`</code>	逻辑或	<code>`a</code>		<code>b`</code>	左真则不算右
<code>!</code>	逻辑非	<code>!flag</code>	—				

3.1.4 赋值运算符（重点）

运算符	等价于	示例
<code>=</code>	—	<code>a = 10</code>
<code>+=</code>	<code>a = a + b</code>	<code>a += 5</code>
<code>-=</code>	<code>a = a - b</code>	<code>a -= 3</code>
<code>*=</code>	<code>a = a * b</code>	<code>a *= 2</code>
<code>/=</code>	<code>a = a / b</code>	<code>a /= 2</code>
<code>%=</code>	<code>a = a % b</code>	<code>a %= 3</code>
结合性： 从右到左，如 <code>a = b = 5</code> 先给 b 赋 5，再给 a 赋 5。		

3.1.5 自增自减运算符（重点）

写法	含义	表达式值
<code>++i</code>	i 先加 1	加之后的值
<code>i++</code>	i 后加 1	加之前的值
<code>--i</code> / <code>i--</code>	同理减 1	同理

3.1.6 位运算符（重点）

运算符	含义	示例		
<code>&</code>	按位与	<code>a & b</code>		
<code> </code>	按位或	<code>a b</code>	<code>`a</code>	<code>b`</code>
<code>^</code>	按位异或	<code>a ^ b</code>		
<code>~</code>	按位取反	<code>~a</code>		
<code><<</code>	左移	<code>a << 2</code> 相当于 $\times 4$		
<code>>></code>	右移	<code>a >> 1</code> 相当于 $\div 2$		

3.1.7 其他运算符（重点）

运算符	名称	示例
<code>? :</code>	条件运算符	<code>max = a > b ? a : b</code>
<code>,</code>	逗号	<code>x = 1, y = 2</code> (值为最后一个)
<code>&</code>	取地址	<code>&x</code>
<code>*</code>	解引用	<code>*p</code>
<code>sizeof</code>	求大小	<code>sizeof(int)</code>
<code>.</code>	结构体成员	<code>stu.name</code>
<code>-></code>	指针访问成员	<code>p->name</code>

3.1.8 运算符优先级（从高到低，常用部分）（重点）

优先级	运算符	结合性		
1	<code>() [] -> .</code>	左→右		
2	<code>! ~ ++ -- * & sizeof</code>	右→左		
3	<code>* / %</code>	左→右		
4	<code>+ -</code>	左→右		
5	<code><< >></code>	左→右		
6	<code>< <= > >=</code>	左→右		
7	<code>== !=</code>	左→右		
8	<code>&</code>	左→右		
9	<code>^</code>	左→右		
10	<code>,</code>	<code>,</code>	左→右	
11	<code>&&</code>	左→右		
12	<code>,</code>		<code>,</code>	左→右
13	<code>? :</code>	右→左		
14	<code>= += -= ...</code>	右→左		
15	<code>,</code>	左→右		

记忆口诀： 单目 → 算术 → 移位 → 关系 → 位运算 → 逻辑 → 条件 → 赋值 → 逗号

3.1.9 常见表达式写法（重点）

场景	写法
交换两数（需临时变量）	<code>t=a; a=b; b=t;</code>
求绝对值	<code>x = x < 0 ? -x : x</code>
判断奇偶	<code>n % 2 == 0</code>
判断闰年	<code>(y%4==0 && y%100!=0)</code>
判断字符是数字	<code>ch >= '0' && ch <= '9'</code>

4 输入输出

大纲

- 1. 格式化输出函数： `printf()` 用于向屏幕输出数据
- 2. 格式化输入函数： `scanf()` 用于从键盘读取数据
- 3. 格式符：
 - o `%d`：整型
 - o `%f`：浮点型 (float/double)
 - o `%c`：字符型
- 4. 取地址符： `scanf()` 读取普通变量时，变量名前必须加 `&` 符号获取地址
- 5. 连续输入： `scanf("%d%d", &a, &b)` 可连续读取多个整数

4.1 知识点总结

4.1.1 printf 格式输出

格式： `printf("格式字符串", 参数1, 参数2, ...);`

格式符	类型	示例
<code>%d</code> / <code>%i</code>	int	<code>printf("%d", 42);</code>
<code>%u</code>	unsigned int	<code>printf("%u", n);</code>
<code>%ld</code> / <code>%lld</code>	long / long long	<code>printf("%lld", x);</code>
<code>%f</code>	float/double	<code>printf("%.2f", 3.14159);</code>
<code>%e</code> / <code>%g</code>	科学计数法	<code>printf("%e", 1.2e3);</code>
<code>%c</code>	char	<code>printf("%c", 'A');</code>
<code>%s</code>	字符串	<code>printf("%s", "hello");</code>
<code>%p</code>	指针地址	<code>printf("%p", p);</code>
<code>%x</code> / <code>%o</code>	十六进制/八进制	<code>printf("%x", 255);</code>
<code>%%</code>	输出 <code>%</code>	<code>printf("100%%");</code>

4.1.2 scanf 格式输入

格式：scanf("格式字符串", &变量1, &变量2);

格式符	类型	注意
<code>%d</code>	int	需传地址 <code>&x</code>
<code>%lf</code>	double	double 用 <code>%lf</code>
<code>%f</code>	float	—
<code>%c</code>	char	注意缓冲区残留 <code>\n</code>
<code>%s</code>	字符串	遇空白停止，需数组名（已是地址）
<code>%99s</code>	限制宽度	防止溢出

4.1.3 输入输出示例

```
#include <stdio.h>
int main(void) {
    int age;
    float height;
    char name[50];

    printf("请输入姓名 年龄 身高: ");
    scanf("%s %d %f", name, &age, &height);
    printf("姓名:%s 年龄:%d 身高:%.2f\n", name, age, height);

    return 0;
}
```

5 流程控制（重点）

大纲

流程控制结构：

1. 顺序结构

2. 选择结构：if、switch

- `if-else`：双分支选择
- `if-else if-else`：多分支选择
- `switch-case`：多分支选择（适合离散值判断）
- **逻辑运算符：**
 - `&&`：逻辑与（两边都为真才为真）
 - `||`：逻辑或（有一边为真即为真）
 - `!`：逻辑非（取反） **switch注意事项：**
- `case` 后必须加 `break` 跳出，否则会继续执行后续case
- `default` 处理未匹配的情况 **条件表达式：** C语言不支持 `0<x<10`，必须写成 `x>0 && x<10`

3. 循环结构：for、while、do-while

三种循环结构：

- `while`：先判断条件，再执行循环体
- `do-while`：先执行循环体，再判断条件（至少执行一次）
- `for`：适合已知循环次数的场景，格式为 `for(初始化; 条件; 更新)`
- **循环控制语句：**
 - `break`：跳出整个循环
 - `continue`：跳过本次循环剩余语句，进入下一次循环
- **嵌套循环：**循环内嵌套循环，外层控制行数，内层控制列数
- **累加/累乘：**初始化累加器/累乘器，循环中逐步累加/累乘

5.1 知识点总结

5.1.1 顺序结构

顺序结构是指程序按顺序执行，没有跳转语句。例如：

```
a = 10;
b = 20;
c = a + b;
```

顺序结构的执行顺序是：先执行a = 10，再执行b = 20，最后执行c = a + b。

5.1.2 选择结构

5.1.3 1 if 语句

形式	语法	说明
单分支	if (条件) 语句;	条件为真执行
双分支	if (条件) 语句1; else 语句2;	二选一
多分支	if ... else if ... else	多条件
复合语句	if (条件) { ... }	多条语句必须加 {}

```
if (score >= 90)
    grade = 'A';
else if (score >= 60)
    grade = 'B';
else
    grade = 'F';
```

5.1.4 2 switch 语句

要点	说明
表达式	必须是整型或枚举
case	后接整型常量表达式
break	通常每个 case 末尾加，防止贯穿
default	可选，所有 case 不匹配时执行

```
switch (op) {
    case '+': result = a + b; break;
    case '-': result = a - b; break;
    default: printf("无效\n"); break;
}
```

5.1.5 3 选择结构速查

需求	推荐		
两路分支	if-else		
多路等值分支	switch		
范围判断	if-else if 链		
多条件组合	if (a && b)、`if (a		b)`

5.1.6 循环结构

5.1.6.1 for 循环

部分	作用
初始化	循环变量初值，只执行一次
条件	每次循环前判断，假则退出
更新	每次循环体后执行

```
for (int i = 0; i < 10; i++)
    printf("%d ", i);
```

5.1.6.2 while 循环

```
while (条件) {
    /* 循环体 */
}
```

特点：先判断后执行，条件一开始为假则一次都不执行。

5.1.6.3 do-while 循环

```
do {
    /* 循环体 */
} while (条件); /* 注意末尾分号 */
```

特点：先执行后判断，至少执行一次。

5.1.6.4 三种循环对比

循环	判断时机	最少执行次数	典型场景
<code>for</code>	前	0	已知次数
<code>while</code>	前	0	未知次数、条件驱动
<code>do-while</code>	后	1	至少执行一次（如菜单）

5.1.6.5 break 与 continue（重点）

语句	作用	适用
<code>break</code>	跳出 当前 循环或 switch	for/while/do-while/switch
<code>continue</code>	跳过本次循环剩余语句，进入下一次	for/while/do-while

5.1.6.6 常见循环模式

场景	写法
累加 1~n	<code>for (i=1; i<=n; i++) sum += i;</code>
遍历数组	<code>for (i=0; i<n; i++) ... a[i] ...</code>
读直到 EOF	<code>while ((ch=getchar()) != EOF) ...</code>
无限循环	<code>while (1) { ... if (条件) break; }</code>
嵌套循环打印图案	外层行，内层列

6 函数

大纲

1. **函数定义**：返回类型、函数名、参数列表、函数体
2. **函数调用**：通过函数名传递参数，获取返回值
3. **参数传递**：默认采用值传递，即传递参数的副本
4. **递归函数**：
 - o 函数调用自身
 - o 必须设置递归终止条件，否则会出现死递归
 - o 每次调用问题规模减小
5. **三目运算符**：`a > b ? a : b` 等价于 if-else 判断
6. **数据类型**：阶乘结果可能很大，需使用 `long long` 类型存储

6.1 知识点总结

6.1.1 函数基本结构

```
返回类型 函数名(参数列表) {  
    /* 函数体 */  
    return 返回值; /* void 函数可省略或写 return; */  
}
```

6.1.2 函数声明与定义

概念	说明	示例
声明（原型）	告诉编译器函数存在	<code>int max(int a, int b);</code>
定义	函数的具体实现	含函数体
调用	使用函数	<code>int m = max(3, 5);</code>

6.1.3 参数传递 (重点)

方式	语法	特点
值传递	<code>void f(int x)</code>	修改形参不影响实参
地址传递	<code>void f(int *x)</code>	可通过指针修改实参
数组传递	<code>void f(int a[], int n)</code>	实际传首元素地址

```
void swap(int *a, int *b) {  
    int t = *a; *a = *b; *b = t;  
}  
/* 调用: swap(&x, &y); */
```

6.1.4 返回值

返回类型	说明
<code>int</code> 、 <code>double</code> 等	<code>return 表达式;</code>
<code>void</code>	无返回值, <code>return;</code> 或直接结束
指针	可返回指针, 不要返回局部变量地址

6.1.5 递归函数

要素	说明
终止条件	必须存在，否则无限递归
递归关系	大问题化为小问题

```
int fact(int n) {
    if (n <= 1) return 1;          /* 终止条件 */
    return n * fact(n - 1);        /* 递归 */
}
```

6.1.6 存储类型

关键字	作用	作用域	生命周期
auto	局部变量默认	块内	函数调用期间
static (局部)	保留值，不重新初始化	块内	整个程序
static (全局)	仅本文件可见	文件内	整个程序
extern	声明外部定义的变量/函数	跨文件	—
register	建议放寄存器（编译器可忽略）	块内	函数调用期间

6.1.7 函数分类速查

类型	示例
无参无返回值	void menu(void);
有参有返回值	int add(int a, int b);
数组作参数	void sort(int a[], int n);
字符串作参数	int len(char s[]);
函数指针参数	void qsort(..., int (*cmp)(...));

6.1.8 main 函数

形式	说明
<code>int main(void)</code>	无命令行参数
<code>int main(int argc, char *argv[])</code>	接收命令行参数
返回值	<code>return 0;</code> 表示成功, 非 0 表示异常

6.1.9 头文件与多文件

文件	内容
<code>xxx.h</code>	函数声明、宏、类型定义
<code>xxx.c</code>	函数实现
使用	<code>#include "xxx.h"</code>
示例:	

```
/* math_utils.h */
int add(int a, int b);
/* math_utils.c */
int add(int a, int b) { return a + b; }
/* main.c */
#include "math_utils.h"
int main(void) { printf("%d\n", add(1, 2)); return 0; }
```

7 指针

大纲

- 1. 指针概念: 地址、指针、取地址、解引用
- 2. 指针声明与使用: 初始化、赋值、解引用、指针移动
- 3. 指针与数组关系: 数组名、指针与数组元素的关系
- 4. 指针运算: 加减、减法、比较
- 5. 常见指针类型: `int *`、`char *`、`void *`、函数指针

7.1 知识点总结

7.1.1 指针概念

概念	说明
地址	内存中每个字节的编号
指针	存放地址的变量
取地址 <code>&</code>	获取变量地址
解引用 <code>*</code>	通过指针访问所指对象

7.1.2 指针声明与使用

写法	含义
<code>int *p;</code>	p 是指向 int 的指针
<code>p = &x;</code>	p 指向 x
<code>*p = 10;</code>	通过 p 修改 x 的值
<code>int *p = NULL;</code>	空指针，不指向有效对象

```
int x = 5;
int *p = &x;
printf("%d\n", *p); /* 输出 5 */
*p = 10;           /* x 变为 10 */
```

7.1.3 指针与数组（重点）

关系	说明
数组名	多数情况下等价于首元素地址
<code>a[i]</code>	等价于 <code>*(a + i)</code>
<code>p[i]</code>	等价于 <code>*(p + i)</code>
指针移动	<code>p+1</code> 移动一个 元素 大小，不是 1 字节

7.1.4 指针运算

运算	说明
<code>p + n</code>	向后移 n 个元素
<code>p - n</code>	向前移 n 个元素
<code>p1 - p2</code>	两指针间元素个数（同数组内）
<code>p1 == p2</code>	比较地址是否相同

7.1.5 常见指针类型

类型	说明
<code>int *p</code>	指向 int
<code>char *p</code>	指向字符，常用于字符串
<code>void *p</code>	通用指针，需强转后使用
<code>int **pp</code>	指向指针的指针
<code>int (*fp)(int)</code>	函数指针

7.1.6 指针作为函数参数

用途	示例
修改实参	<code>void swap(int *a, int *b)</code>
传递数组	<code>void func(int *arr, int n)</code>
返回多个结果	通过指针参数写回
传递大结构体	传地址避免复制

7.1.7 指针与 const（难点）

写法	含义
<code>const int *p</code>	指向常量的指针，*p 不可改
<code>int *const p</code>	指针常量，p 不可改指向
<code>const int *const p</code>	两者都不可改

8 数组

大纲

- 1. 数组特性：
 - 下标从0开始
 - 连续内存存储
 - 数组名代表首元素地址
- 2. 二维数组：
 - 行为主序存储（按行存放）
 - 定义格式：`int arr[row][col]`
- 3. 数组操作：
 - 遍历：使用for循环访问每个元素
 - 最值查找：初始化后遍历比较
- 4. 冒泡排序：
 - 相邻元素比较交换
 - 每轮将最大元素"冒泡"到末尾
 - 时间复杂度： $O(n^2)$
- 5. 函数参数：数组作为参数传递时，实际传递的是首地址

8.1 知识点总结

8.1.1 一维数组

写法	说明	示例
定义	<code>类型 名[长度];</code>	<code>int a[10];</code>
初始化	定义时赋初值	<code>int a[5] = {1,2,3,4,5};</code>
部分初始化	其余为 0	<code>int a[5] = {1,2};</code> → 1,2,0,0,0
省略长度	由初值个数决定	<code>int a[] = {1,2,3};</code> → 长度 3
全零	常用写法	<code>int a[10] = {0};</code>

8.1.2 数组访问

规则	说明
下标	从0 开始
合法范围	0 ~ 长度-1
越界	C 不检查，可能导致未定义行为

```
int a[5] = {10, 20, 30, 40, 50};
printf("%d\n", a[0]);    /* 10 */
printf("%d\n", a[2]);    /* 30 */
a[4] = 100;              /* 修改最后一个元素 */
```

8.1.3 二维数组

写法	说明
定义	类型 名[行][列];
初始化	按行赋初值
元素个数	行 × 列
存储	行优先（先存第一行）

```
int a[2][3] = {{1,2,3}, {4,5,6}};
printf("%d\n", a[1][2]);    /* 6 */
```

8.1.4 数组与 sizeof

表达式	含义（一维 int a[10]）
sizeof(a)	整个数组字节数（40）
sizeof(a[0])	单个元素字节数（4）
sizeof(a)/sizeof(a[0])	元素个数（10）

8.1.5 数组作为函数参数

```
void printArray(int a[], int n) {
    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);
}

int main(void) {
    int arr[] = {1, 2, 3, 4, 5};
    printArray(arr, 5);
    return 0;
}
```

要点	说明
传参退化	数组名传参变为指针
必须传长度	函数内无法 sizeof 得元素个数
二维数组	需指定列数: <code>void f(int a[][4], int rows)</code>

8.1.6 常见操作

操作	代码思路
遍历	<code>for (i=0; i<n; i++)</code>
求和	累加 <code>a[i]</code>
求最大	设 <code>max=a[0]</code> , 逐个比较
逆序	首尾交换, 向中间靠拢
查找	线性遍历或二分 (有序)
复制	逐个赋值或 <code>memcpy</code>

8.1.7 数组与指针

表达式	类型/含义
<code>a</code>	指向首元素的指针
<code>a + i</code>	第 <code>i</code> 个元素地址
<code>*(a + i)</code>	等价 <code>a[i]</code>
<code>&a[i]</code>	第 <code>i</code> 个元素地址

8.1.8 字符数组 vs 字符指针

	<code>char s[] = "hi";</code>	<code>char *s = "hi";</code>
存储	栈上数组，内容可改	指针指向字符串常量
修改	<code>s[0]='H'</code> 合法	修改内容非法
sizeof	数组大小	指针大小

9 字符串

大纲

- 1. **字符串本质**：字符数组，以 `\0`（ASCII码0）作为结束标志
- 2. **字符串存储**：实际占用空间 = 字符数 + 1（结束符）
- 3. **字符串输入**：`scanf("%s", str)` 自动在末尾添加 `\0`
- 4. **字符串遍历**：通过索引访问，直到遇到 `\0` 为止
- 5. **标准库函数**：`string.h` 提供 `strlen()`、`strcpy()`、`strcmp()` 等函数

9.1 知识点总结

9.1.1 字符串本质

要点	说明
本质	以 <code>\0</code> 结尾的 <code>char</code> 数组
字面量	<code>"Hello"</code> 含 6 字节：H e l l o \0
结束符	<code>'\0'</code> ，ASCII 值为 0
头文件	<code>#include <string.h></code>

9.1.2 字符串定义方式

写法	说明
<code>char s[] = "hello";</code>	字符数组，可修改
<code>char s[20] = "hello";</code>	指定大小，剩余为 <code>\0</code>
<code>char *s = "hello";</code>	指向字符串常量， 不可修改
<code>char s[20];</code>	未初始化，需后续赋值

9.1.3 字符串输入

函数	用法	特点
<code>scanf("%s", s)</code>	读无空格字符串	遇空白停止， 不安全 （需控制长度）
<code>fgets(s, size, stdin)</code>	读一行	安全，含 <code>\n</code> ，推荐
<code>gets(s)</code>	已废弃	禁止使用

```
char s[100];
fgets(s, sizeof(s), stdin);
s[strcspn(s, "\n")] = '\0'; /* 去掉换行 */
```

9.1.4 字符串输出

函数	格式符/用法
<code>printf("%s", s)</code>	输出到 <code>\0</code>
<code>puts(s)</code>	输出并换行
<code>fputs(s, stdout)</code>	输出到指定流

9.2 string.h 常用函数

函数	原型	功能	返回值
<code>strlen</code>	<code>size_t strlen(const char *s)</code>	求长度（不含 <code>\0</code> ）	字符个数
<code>strcpy</code>	<code>char *strcpy(char *dst, const char *src)</code>	复制	dst
<code>strncpy</code>	<code>char *strncpy(..., size_t n)</code>	最多复制 n 个	dst
<code>strcat</code>	<code>char *strcat(char *dst, const char *src)</code>	连接	dst
<code>strcmp</code>	<code>int strcmp(const char *a, const char *b)</code>	比较	0 相等，>0 或 <0
<code>strchr</code>	<code>char *strchr(const char *s, int c)</code>	找字符	首次出现位置
<code>strstr</code>	<code>char *strstr(const char *s, const char *sub)</code>	找子串	首次出现位置

9.2.1 字符串比较规则

strcmp 返回值	含义
<code>0</code>	两串相等
<code>> 0</code>	第一个更大（按字典序）
<code>< 0</code>	第一个更小

注意： 不要用 `==` 比较字符串，那是比较地址。

9.2.2 字符处理 (ctype.h)

函数	功能
<code>isalpha(c)</code>	是否字母
<code>isdigit(c)</code>	是否数字
<code>isupper(c)</code> / <code>islower(c)</code>	是否大/小写
<code>isspace(c)</code>	是否空白
<code>toupper(c)</code> / <code>tolower(c)</code>	转大/小写

9.2.3 手动实现常用操作

操作	思路
求长度	<code>while (*s++) n++;</code>
复制	<code>while ((*d++ = *s++));</code>
比较	逐字符比，遇 <code>\0</code> 或不等则停
反转	首尾指针交换

10 结构体

大纲

- 1. **结构体定义：** `struct` 关键字定义自定义数据类型
- 2. **typedef：** 为结构体类型定义别名，简化使用
- 3. **结构体成员访问：** 使用 `.` 运算符访问成员
- 4. **结构体变量声明：** `Student s;` 创建结构体变量
- 5. **内存布局：** 结构体成员按定义顺序存储，可能存在内存对齐

10.1 知识点总结

10.1.1 定义与使用

```
struct Student {
    int id;
    char name[32];
    float score;
};

struct Student s1;
s1.id = 1001;
strcpy(s1.name, "Tom");
s1.score = 90.5;
```

操作	语法
定义变量	struct Student s;
初始化	struct Student s = {1001, "Tom", 90.5};
访问成员	s.id、s.name
指针访问	p->id 等价 (*p).id

10.1.2 typedef 简化

```
typedef struct {
    int x, y;
} Point;

Point p = {10, 20}; /* 无需写 struct */
```

10.1.3 结构体数组

```
struct Student st[3];
st[0].id = 1;
/* 或初始化 */
struct Student st[] = {
    {1, "A", 80},
    {2, "B", 90}
};
```

10.1.4 结构体作为函数参数

方式	特点
值传递	复制整个结构体，大结构体效率低
指针传递	<code>void f(struct Student *p)</code> ，推荐

10.1.5 结构体对齐（了解）

概念	说明
对齐	成员按一定字节边界存放
sizeof	可能大于各成员之和
原因	CPU 访问对齐地址更高效

10.1.6 嵌套结构体

```
typedef struct {
    int year, month, day;
} Date;

typedef struct {
    char name[32];
    Date birthday;
} Person;

Person p;
p.birthday.year = 2000;
```

11 文件操作

大纲

- 1. **文件指针**: `FILE *fp` 指向文件的指针
- 2. **文件打开**: `fopen(filename, mode)`
 - o `"r"`: 只读模式
 - o `"w"`: 写入模式（覆盖原有内容）
 - o `"a"`: 追加模式
- 3. **文件关闭**: `fclose(fp)` 关闭文件，释放资源
- 4. **文件写入**: `fprintf(fp, format, ...)` 格式化写入
- 5. **文件读取**: `fgets(buf, size, fp)` 按行读取
- 6. **错误处理**: 检查 `fopen` 返回值是否为NULL

11.1 知识点总结

11.1.1 文件指针

头文件: `#include <stdio.h>`

```
FILE *fp;                /* 文件指针类型 */
fp = fopen("data.txt", "r"); /* 打开文件 */
/* ... 读写操作 ... */
fclose(fp);              /* 关闭文件, 必须调用 */
```

11.1.2 fopen 打开模式

模式	含义	文件不存在	文件存在
"r"	只读	失败	从头读
"w"	只写	创建	清空后写
"a"	追加	创建	末尾追加
"r+"	读写	失败	从头读写
"w+"	读写	创建	清空后读写
"a+"	读+追加	创建	末尾追加
"rb" "wb" 等	二进制模式	同上	同上

返回值: 成功返回 `FILE*`, 失败返回 `NULL`, 务必检查。

```
FILE *fp = fopen("test.txt", "r");
if (fp == NULL) {
    perror("打开失败");
    return 1;
}
```

11.1.3 字符级读写

函数	功能	返回值
<code>fgetc(fp)</code>	读一个字符	字符或 EOF
<code>fputc(c, fp)</code>	写一个字符	字符或 EOF
<code>getc(fp)</code>	同 <code>fgetc</code>	—
<code>putc(c, fp)</code>	同 <code>fputc</code>	—

11.1.4 字符串/行读写

函数	功能
<code>fgets(s, n, fp)</code>	读一行（最多 n-1 字符）
<code>fputs(s, fp)</code>	写字符串

11.1.5 格式化读写

函数	功能
<code>fprintf(fp, fmt, ...)</code>	格式化写入文件
<code>fscanf(fp, fmt, ...)</code>	格式化从文件读取

```
fprintf(fp, "%d %.2f %s\n", id, score, name);
fscanf(fp, "%d %f %s", &id, &score, name);
```

11.1.6 块读写（二进制）

函数	功能
<code>fread(ptr, size, count, fp)</code>	读 count 个 size 字节
<code>fwrite(ptr, size, count, fp)</code>	写 count 个 size 字节

```
int a[10];
fwrite(a, sizeof(int), 10, fp);
fread(a, sizeof(int), 10, fp);
```

11.1.7 文件定位

函数	功能
<code>fseek(fp, offset, origin)</code>	移动文件指针
<code>ftell(fp)</code>	返回当前位置
<code>rewind(fp)</code>	回到文件开头

origin: `SEEK_SET`（开头）、`SEEK_CUR`（当前）、`SEEK_END`（末尾）

11.1.8 文件状态检测

函数	功能
<code>feof(fp)</code>	是否到达文件末尾
<code>ferror(fp)</code>	是否发生错误
<code>clearerr(fp)</code>	清除错误和 EOF 标志

11.1.9 文本文件 vs 二进制文件

	文本模式	二进制模式
打开	<code>"r"</code> <code>"w"</code>	<code>"rb"</code> <code>"wb"</code>
内容	人类可读	原始字节
换行	可能转换	不转换
适用	配置文件、日志	结构体、图片、音频

11.1.10 完整示例：复制文件

```
#include <stdio.h>
int main(void) {
    FILE *in = fopen("src.txt", "r");
    FILE *out = fopen("dst.txt", "w");
    if (!in || !out) return 1;

    int ch;
    while ((ch = fgetc(in)) != EOF)
        fputc(ch, out);

    fclose(in);
    fclose(out);
    return 0;
}
```

11.1.11 完整示例：结构体存盘

```
typedef struct { int id; char name[20]; } Record;
Record r = {1, "Tom"};
FILE *fp = fopen("data.bin", "wb");
fwrite(&r, sizeof(Record), 1, fp);
fclose(fp);
```

11.1.12 文件操作注意事项

要点	说明
检查 fopen	失败时返回 NULL
必须 fclose	否则可能丢数据、泄漏资源
w 模式	会清空已有文件
路径	Windows 可用 <code>"C:\\data.txt"</code> 或 <code>"C:/data.txt"</code>
权限	确保有读写权限

12 预处理

12.1 处理指令概览

指令	作用
<code>#include</code>	包含头文件
<code>#define</code>	宏定义
<code>#undef</code>	取消宏定义
<code>#if</code> / <code>#ifdef</code> / <code>#ifndef</code>	条件编译
<code>#else</code> / <code>#elif</code> / <code>#endif</code>	条件分支
<code>#pragma</code>	编译器相关（实现依赖）
<code>#error</code>	编译报错并输出信息

特点：以 `#` 开头，不需要分号结尾。

12.2 文件包含

写法	搜索顺序
<code>#include <stdio.h></code>	系统目录（标准库）
<code>#include "myfile.h"</code>	当前目录 → 系统目录

常见标准头文件：

头文件	内容
<code>stdio.h</code>	输入输出
<code>stdlib.h</code>	内存分配、转换、exit
<code>string.h</code>	字符串函数
<code>math.h</code>	数学函数
<code>ctype.h</code>	字符处理
<code>time.h</code>	时间日期
<code>stdbool.h</code>	bool 类型 (C99)

12.3 定义 #define

12.3.1 常量宏

```
#define PI 3.14159
#define MAX 100
```

12.3.2 带参宏

```
#define MAX(a, b) ((a) > (b) ? (a) : (b))
#define SQUARE(x) ((x) * (x))
```

注意	说明
加括号	防止运算符优先级问题
副作用	避免 <code>#define SQ(x) x*x</code> 在 <code>SQ(i++)</code> 时出错
无类型检查	宏只是文本替换

12.3.3 宏 vs const

	<code>#define</code>	<code>const</code>
处理阶段	预处理	编译
类型	无	有
调试	难	易
推荐	条件编译、简单常量	一般常量

12.4 条件编译

```
#define DEBUG 1

#ifdef DEBUG
    printf("调试信息\n");
#endif

#ifndef HEADER_H
#define HEADER_H
/* 头文件内容 */
#endif
```

指令	含义
<code>#ifdef MACRO</code>	宏已定义则编译
<code>#ifndef MACRO</code>	宏未定义则编译（头文件保护）
<code>#if 表达式</code>	表达式非 0 则编译
<code>#else</code>	否则分支
<code>#endif</code>	结束条件块

12.5 预处理技巧

场景	写法
头文件保护	<code>#ifndef XXX_H / #define XXX_H / #endif</code>
调试开关	<code>#ifdef DEBUG ... #endif</code>
跨平台	<code>#ifdef _WIN32 ... #else ...</code>
字符串化	<code>#define STR(x) #x → STR(hello) → "hello"</code>
连接符号	<code>#define CAT(a,b) a##b</code>

12.6 预定义宏

宏	含义
<code>__FILE__</code>	当前源文件名
<code>__LINE__</code>	当前行号
<code>__DATE__</code>	编译日期
<code>__TIME__</code>	编译时间
<code>__func__</code>	当前函数名 (C99)

```
printf("错误: %s 第 %d 行\n", __FILE__, __LINE__);
```

12.7 pragma 示例

```
#pragma once /* 部分编译器: 头文件只包含一次 */
#pragma warning(disable:4996) /* MSVC: 忽略某警告 */
```

12.8 预处理执行顺序

1. 处理 `#include`、展开 `#define`
2. 处理条件编译，删除不满足条件的代码
3. 将结果交给编译器